Assignment 4: Abstraction; Constructors

Goals: Learn to use custom constructors, determine equality via dynamic-dispatch, and determine non-structural equality.

Instructions

Be very, very careful with naming! Again, the solution files expect your submissions to be named a certain way, so that they can define their own `Examples` class with which to test your code. Therefore, whenever the assignment specifies:

- the names of classes,
- the names and types of the fields within classes,
- the names, types and order of the arguments to the constructor,
- the names, types and order of arguments to methods, or
- filenames,

...be sure that your submission uses exactly those names.

Make sure you follow the style guidelines that Bottlenose enforces. For now the most important ones are: using spaces instead of tabs, indenting by 2 characters, following the naming conventions (data type names start with a capital letter, names of fields and methods start with a lower case letter), and having spaces before curly braces.

You will submit this assignment by the deadline using the Bottlenose submission system. You may submit as many times as you wish. Be aware of the fact that close to the deadline the Bottlenose system may slow down to handle many submissions - so try to finish early.

There will be a separate submission for each problem - it makes it easier to grade each problem, and to provide you with the feedback for each problem you work on.

The submissions will be organized as follows:

Practice Problems

Work out these problems on your own. Save them in an electronic portfolio, so you can show them to your instructor, review them before the exam, use them as a reference when working on the homework assignments.

- Problems 18.1 - 18.4 on page 225
- Problem 18.5 on page 229
- Problem 18.6 on page 234
- Problem 19.4 on page 263

Problem 1: Working with Custom Constructors, A Taste of Equality

A `BagelRecipe` can be represented as the amount of flour, water, yeast, salt, and malt in the recipe. A perfect bagel results when the ratios of the weights are right:

- the weight of the flour should be equal to the weight of the water
- the weight of the yeast should be equal the weight of the malt
- the weight of the salt + yeast should be 1/20th the weight of the flour

Design the `BagelRecipe` class. The fields should be of type `double` and represent the weight of the ingredients as ounces. Provide three constructors for this class:

- Your main constructor should take in all of the fields and *enforce* all above constraints to ensure a perfect bagel recipe.
- Provide another constructor that only requires the weights of flour and yeast, and produces a perfect bagel recipe.
- Provide another constructor that takes in the flour, yeast and salt **as volumes rather than weights**, and tries to produce a perfect recipe. Here, too, a perfect recipe should be enforced.

Flour and water volumes are measured in cups, while yeast, salt, and malt volumes are measured in teaspoons.

Helpful conversion factors:

- 48 teaspoons = 1 cup
- 1 cup of yeast = 5 ounces
- 1 cup of salt = 10 ounces
- 1 cup of malt = 11 ounces
- 1 cup of water = 8 ounces
- 1 cup of flour = 4 and 1⁄4 ounces

You should use an `IllegalArgumentException` with a helpful message if the above constraints cannot be enforced.

Once you've completed the above constructors, you should:

- Remove as much duplicate code as possible from these constructors.
- Implement the method `sameRecipe(BagelRecipe other)` which returns true if the same ingredients have the same weights to within 0.001 ounces.

Submit your work in a file named `Bagel.java`.

Problem 2: Abstracting over Data Definitions

> Related files:
> Entertainment.java

Entertainment media are elements of the media that focus on delivering entertainment to the general public. These include magazines, television series, and podcasts. Every medium has a name and price. Different kinds of entertainment can also contain additional information as shown in the supplied `Entertainment.java` file.

Note: only the `totalPrice()` method is properly implemented. The others are *stubs* that currently return a dummy value, so the code will compile but not yet work.

Warmup: Download the file and work out the following problems:

- Make one more example of data for each of the three classes and add more tests for the `totalPrice` method (that is already defined) using them.
- Design the method `duration` which produces an `int` with the number of minutes

this entertainment provides. We assume a magazine provides 5 minutes of entertainment per page and TV series and podcasts provide 50 minutes of entertainment per episode.

- Design the method `format` which produces a `String` showing the name and price of this entertainment. The name and price should be separated by a comma and a space and end with a period.

- Design the method `sameEntertainment` that determines whether the this entertainment is the same as the given one.

Once you have finished these methods and are confident that they work properly, save the work you have done to a separate file. Do not submit the code as written so far. The problems below are the main point of this exercise, and it will be helpful for you to preserve the code written so far as a reference against which to compare your revised code below. *Again, submit only the work below.*

- Look at the code and identify all places where the code repeats — the opportunity for abstraction.

  Lift the common fields to an abstract class `AEntertainment`.

  Make sure you include a constructor in the abstract class, and change the constructors in the derived classes accordingly. Run the program and make sure all test cases work as before.

- For each method that is defined in all three classes decide to which category it belongs:

  - The method bodies in the different classes are all different, and so the method has to be declared as `abstract` in the `abstract` class.

  - The method bodies are the same in all classes and it can be implemented concretely in the `abstract` class.

  - The method bodies are the same for two of the classes, but are different in one class — therefore we can define the common body in the `abstract` class and override it in only one derived class.

  Now, lift the methods that can be lifted and run all tests again.